Machine Learning

Project of Sentiment Analysis

Francesco Astolfi

Laurea Magistrale in Ingegneria Informatica – A.Y. 2024/25

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Dipartimento di Ingegneria Civile e Ingegneria Informatica

The task

2.2 Traccia T2 - Sentiment Analysis: Twitter

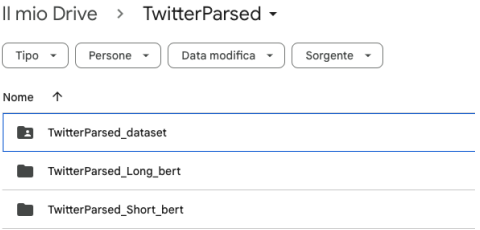
L'obiettivo del progetto è sviluppare un modello di ML per classificare messaggi pubblicati sul social Twitter come *positivi* o *negativi*. Il dataset di riferimento, che contiene tweet in lingua inglese, è `TwitterParsed.zip`.

Valutare l'accuratezza dei modelli su dati di validazione opportunamente estratti dal dataset, provando almeno 2 diverse configurazioni del modello (e.g., variando lunghezza dei frammenti di testo in input, o la dimensione del vocabolario). Inoltre, verificare il funzionamento del modello su messaggi scritti da voi.

The project : link to the project

Dependencies

The following directory need to be loaded on the drive account that want to start the project drive-directory .
This directory is essential because it contains the fine-tuned model and the dataset required for the project. Ensure that the drive is properly configured to access these resources.



About the project

- **What is the sentiment analysis?**

Sentiment analysis is the task of determining whether a given sentence expresses a positive or negative sentiment. In this specific case, the sentences are tweets. This task is particularly challenging because it requires a deep understanding of the semantics and context of the sentence. This is why BERT is used.

- **Why it's used Bert?**

BERT is a powerful model in the field of Natural Language Processing (NLP). It is primarily used to generate contextual embeddings of the input text. These embeddings capture the meaning of words in context, making them highly effective for various downstream tasks.
For example: Sentiment analysis, Topic classification,
Fill-in-the-blank tasks (e.g., masked language modeling)…
However, BERT is not designed for tasks that require text generation, such as question answering or text translation. For such tasks, models with both encoder and decoder components (e.g., Transformer-based architectures) or
decoder-only models (e.g., GPT) are more suitable.

Steps in the project

- Loading of the libraries
- Loading of the dataset and the extraction from the zip file
- Splitting of the data
- Selecting of the model
- Defining of the model structure
- Fine- tuning
- See the performance of the model

Loading of the libraries

The imported libraries are essential for:

- Working with deep learning models.
- Managing transformer-based models, including preprocessing tasks.
- Handling and organizing files within directories.

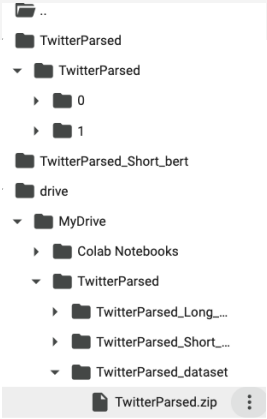
- **tensorflow-text:** Used for text preprocessing.
- **tf-models-official:** Used to leverage pre-trained models.
- **tensorflow:** Used for manipulating and building deep learning models.
- **tensorflow_text:** The main library for deep learning tasks involving text.
- **official.nlp:** Used to apply optimization methods like AdamW.
- **os:** Used for performing operations on the file system.
- **zipfile :** Used for extract the dataset.zip
- **shutil:** Used for performing operations on files, like copy directory
- **pandas:** Used for load the data and use the stratify split
- **sklearn.model_selection :** Used for the split of the data
- **matplot :** For visualizing the performance of the model during the history
- **logger:** Used to display only important errors during output.

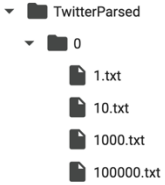
1]

```
!pip install tensorflow-text==2.14.*
!pip install "tf-models-official==2.14.*"
import tensorflow as tf
from tensorflow import keras
import tensorflow_hub as hub
import tensorflow_text as text
from tensorflow.data import AUTOTUNE
from official.nlp import optimization
from tensorflow.keras.callbacks import EarlyStopping
import os
import zipfile
from google.colab import drive
import shutil
import pandas as pd

from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt # for plotting the performance between training and validation data
tf.get_logger().setLevel('ERROR')
```

The dataset



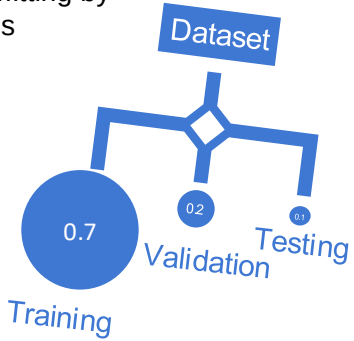


The original data is provided as a zip file in the directory of drive and automatically loaded in other way, without drive, the zip must be added every new runtime.
The data is organized into two subdirectories, where each directory acts as a label.
Inside these directories, there are .txt files containing individual tweet instances.

Splitting the data

The data split is a crucial step in this project to ensure that not all the data is used at once. In this case, the data is divided into three sets:

- Training Data: Used for fine-tuning the model.
- Validation Data: Used to evaluate the model during training and to implement early stopping. Early stopping is particularly useful if the training process takes too long, as it prevents overfitting by stopping training after a few epochs if no improvement is observed.
- Test Data: Used to evaluate the model on unseen data, providing an unbiased assessment of its performance.

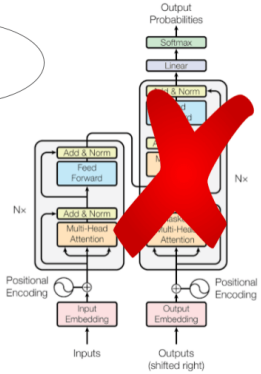


Selecting the model

The model was chosen for experimental purposes. While it may not represent the best possible performance, it's used as a solid starting point for working with transformers.
The model was downloaded from TensorFlow because training a model from scratch requires significant effort and resources. Instead, it's opted to use a pre-trained model and then applied fine-tuning to adapt it to our specific task.

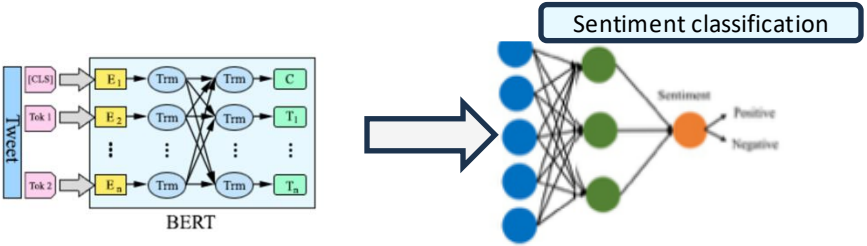
TensorFlow

https://www.tensorflow.org/



Defining the model structure

BERT does not process raw text autonomously; the text must be preprocessed and encoded beforehand.
The preprocessing involves several steps, such as cleaning the data, tokenizing it add the padding or truncate it.
The encoder uses the preprocessed data to generate a tensor from the tokens. These tokens are then passed to BERT, which processes them and produces a cumulative, contextually significant embedding for the entire sentence. This embedding is subsequently used for classification within the neural network.

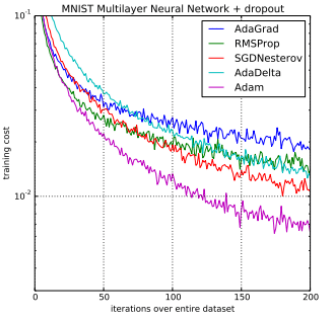


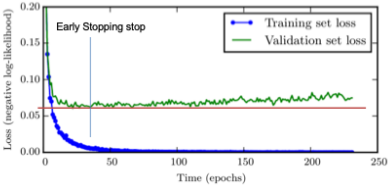
Fine Tuning

Fine-tuning the model is a computationally intensive process. Training the model for just 5 epochs takes a significant amount of time. During fine-tuning, the following configurations were applied:

- A starting learning rate was set to control the step size during optimization.
- An early stopping algorithm was implemented to prevent overfitting by monitoring validation performance and stopping training if no improvement is observed.
- The AdamW optimizer that consent to use a mobile exponential average to efficiently update the model's weights while incorporating weight decay for regularization.

This setup ensures that the model is fine-tuned effectively while managing computational resources and avoiding overfitting.





Metrics evaluated

Loss on the cross entropy

$$\mathcal{L} = -(y \cdot \log(p) + (1-y) \cdot \log(1-p))$$

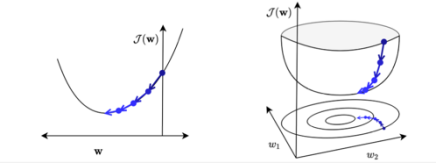
Y: True label (1 = positive class, 0 = negative class).

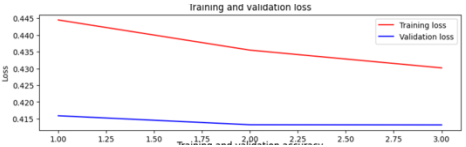
p: Predicted probability of belonging to the positive class.

When classifying an instance as **y = 1**, the prediction should be penalized if it deviates significantly from 1. If **p** is close to 1, the logarithmic loss approaches 0, indicating a correct prediction. When **p** is close to 1, the loss approaches 0, indicating accurate predictions.

Larger deviations require greater corrections.

A loss near 0 means the model performs well, with no further corrections needed.





Metrics evaluated

Binary accurancy

Confusion Matrix:

- Columns: Predicted values. Rows: Real values.
- Diagonal: Correct predictions.

Accuracy:

- Calculated as: Correct predictions / Total predictions.
- Close to 1 indicates high performance.

Note:

- Accuracy is meaningful only when there's a good balance between positive and negative examples.

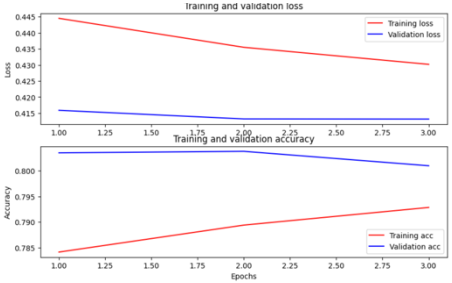
		Predicted	
		Spam	Non-spam
Actual	Spam	**600** (True positive)	**300** (False negative)
	Non-spam	**100** (False positive)	**9000** (True negative)

$$\text{Accurancy} = \frac{correct\ predictions}{all\ the\ predictions}$$

Performance on the validation data

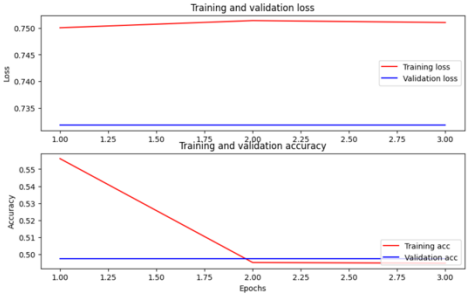
The fine-tuned model, which was recursively refined and saved in the repository on Google Drive, after the cycle of something like 6 epoques . The performance of the models accurancy and loss during the training history is as follows:

Performance of shorter model



Short model get better performance through the last 3 epoques.

Performance of extended model



Extented model go to overfit the data and get worst performance. The early stopping doesn't start because is setted with 3 epoques of patience.

Performance on the test data

The fine-tuned model, which was recursively refined and saved in the repository on Google Drive, after the cycle of something like 6 epoques .

The performance of the models on the test data :

```python
def build_classifier_model(config):
    # Definizione della configurazione
    if config == "config_1":
        max_length = 128
        vocab_size = 30000
    else:
        max_length = 256
        vocab_size = 50000
```

```
✅ Model config_1 already exists. Loading...
Model loaded successfully.
75/75 [==============================] - 83s 1s/step - loss: 0.4114 - binary_accuracy: 0.8033
Loss: 0.4114032983779907
Accuracy: 0.8032535314559937
```

Performance of shorter model

Performance of extended model

```
✅ Model config_2 already exists. Loading...
Model loaded successfully.
75/75 [==============================] - 81s 1s/step - loss: 0.7307 - binary_accuracy: 0.4979
Loss: 0.7307448983192444
Accuracy: 0.4978998601436615
```

Performance on the manually input data

The fine-tuned model, which was recursively refined and saved in the repository on Google Drive, after the cycle of something like 6 epoques .

The performance of the models on the manually input data :

```
✅ Model config_1 already exists. Loading...
Model loaded successfully.
✅ Model config_2 already exists. Loading...
Model loaded successfully.
Results from the model in memory from short :
input: this is such an amazing movie! : score: 0.985822
input: The movie was great!           : score: 0.986416
input: The movie was meh.             : score: 0.491039
input: The movie was ok. .            : score: 0.901429
input: The movie was terrible...      : score: 0.024347
input: Just had the most incredible day! 🤯 Started with a refres

Results from the model in memory from long :
input: this is such an amazing movie! : score: 0.568583
input: The movie was great!           : score: 0.428307
input: The movie was meh.             : score: 0.416650
input: The movie was ok. .            : score: 0.413185
input: The movie was terrible...      : score: 0.308136
input: Just had the most incredible day! 🤯 Started with a refres
```

Performance of shorter model

```
...  ıde #PositiveVibes #LifeIsGood : score: 0.986664
```

Performance of extended model

```
...  ıde #PositiveVibes #LifeIsGood : score: 0.290977
```

Conclusions

Its seems that the shorter model with have more good performance on restricted proposition but when it’s become more larger the larger one have more good performance. The other fact is that there are to loss example and the larger model go to overfit them.